

Obsidian, Networked Knowledge, and AI-Augmented Note Tooling

Executive summary

The strongest conclusion from the research is that **Obsidian fits one important aspect of human knowledge particularly well: semantic knowledge is associative rather than purely sequential**. Cognitive science frequently models semantic memory as networks of concepts and associations, and experimental work finds that the structure and connectedness of those associations relates to retrieval, creativity, insight, and memory. At the same time, human memory is **not simply a graph**: episodic memory strongly preserves temporal, spatial, causal, and narrative context. A knowledge system that is only topical can therefore lose information that chronological notes preserve. ¹

The most defensible design is consequently **hybrid**: capture events sequentially when sequence carries meaning, then distill durable concepts into topical notes connected by explicit links, while preserving links back to the chronological sources. This combines the “external storage” function of notes with semantic restructuring rather than pretending that either a notebook or a graph is a complete model of cognition. Classic note-taking research distinguishes the act of encoding information while taking notes from the value of the notes as an external store for later review. ²

Finding	Research conclusion
Topical/ networked notes	Best suited to reusable concepts, research synthesis, cross-project knowledge, literature notes, evergreen reference material, and discovering relationships. Semantic-network research provides a plausible cognitive rationale, while concept-map meta-analyses provide direct learning evidence for constructing and using explicit conceptual relationships. ³
Sequential/linear notes	Best suited to meetings, lectures, lab notebooks, project logs, interviews, case histories, procedures, and narratives, where chronology, causation, provenance, or “what happened next?” is itself information. Episodic-memory research explicitly emphasizes associations between items and their spatiotemporal context. ⁴
Networked is not automatically better	Hypertext can impose navigation and working-memory costs, and retrieval practice can outperform concept mapping for learning. The effective ingredient is active organization/retrieval, not merely drawing links. ⁵
Obsidian's distinctive architecture	A vault is fundamentally a filesystem folder; notes are Markdown text files, metadata can live in YAML properties, and links/backlinks/graph views are indexes over those files. The public plugin API exposes the vault, workspace, metadata cache, views, and editor. ⁶

Finding	Research conclusion
Obsidian itself is not open source	Its application code remains proprietary/copyrighted even though users own their note content and the plugin API/file formats enable substantial interoperability. ⁷
Closest OSS analogues	Foam most closely reproduces Obsidian-style Markdown/wikilink/backlink/graph semantics inside VS Code; SilverBullet is particularly attractive as a fully open programmable Markdown system; Logseq is stronger for block/outliner workflows; SiYuan and Trilium are powerful but use materially different internal data models; Joplin is exceptionally mature but database/notebook-oriented. ⁸
AI direction	The ecosystem has moved beyond “chat with a note.” Current projects support semantic retrieval, local models, tool-using agents, file creation, research workflows, and autonomous multi-step work. Copilot for Obsidian V4, for example, exposes opencode, Claude Code, and Codex-style agents inside the vault with controllable permissions. ⁹
Best AI formatter design	Use deterministic Markdown parsing/linting for syntax, an LLM only for semantic restructuring, strict preservation rules for Obsidian syntax, structural validation, a visible diff, and explicit approval before writes. Obsidian Linter is a useful model for the deterministic half of this pipeline. ¹⁰

One caveat is important throughout this report: the academic literature does **not** generally study a formal binary called “topical versus sequential note-taking,” nor did I find controlled evidence that Obsidian itself improves memory. The evidence is indirect but convergent, coming from semantic-memory networks, concept mapping, hypertext research, episodic memory, retrieval practice, and note-taking studies.

Topical and sequential note-taking: definitions and theory

What the two approaches actually mean

“Topical” and “sequential” are better understood as **information architectures** than mutually exclusive note-taking methods.

A **sequential note system** derives much of its structure from position. Entry B follows entry A because it occurred later, appeared later in a lecture, or was written on the next page. A chronological project log such as:

```

2026-08-17
- Customer reported timeout
- Reproduced after deploy
- Rolled back version 4.2
- Latency recovered

```

2026-08-18

- Root cause: connection-pool regression

encodes valuable information simply by retaining order.

A **topical or networked system** instead tries to make the unit of organization correspond to a relatively persistent subject or idea:

```
[[Connection pool exhaustion]]
  ↓
[[Database latency]]
  ↓
[[Timeout failure modes]]
  ↓
[[Release 4.2 incident]]
```

The same incident can then contribute evidence to several reusable concepts.

This distinction maps reasonably well onto two different things human memory must accomplish.

Semantic memory represents facts, concepts, categories, and associations, and network models have long treated concepts as nodes connected by relations. Modern semantic-network research continues to operationalize semantic memory in essentially this way. ¹¹ **Episodic memory**, by contrast, binds events to temporal and spatial context; recent hippocampal work explicitly characterizes episodic memory in terms of items associated with their spatiotemporal context. ¹²

That leads to the first major design principle:

Human knowledge is partly associative and partly sequential. A good knowledge-management system should preserve both.

Why the network metaphor is cognitively plausible

Collins and Loftus's influential spreading-activation model proposed that retrieving one concept facilitates access to related concepts through an associative semantic network. It is a model of cognition rather than a claim that neurons literally contain Markdown wikilinks, but it gives a theoretical basis for interfaces in which following one idea brings nearby ideas into view. ¹³

Contemporary network-science research makes this connection more measurable. Ovando-Tellez and colleagues describe semantic-memory networks in which nodes are concepts and edges reflect relationships; differences in network organization have been associated with creative ability. ¹⁴ Wulff and colleagues likewise note that semantic-network content and connectivity are experience-dependent and vary across individuals, which is an important warning against assuming that there is one objectively correct graph for everyone. ¹⁵

Network structure also appears during learning and insight. Research on problem solving has found that restructuring semantic-memory networks accompanies successful insight, while work on naturalistic stories found that events with greater semantic or causal connectedness were more likely to be remembered. ¹⁶

This is where topical notes have their strongest conceptual advantage. A file called `Connection pooling.md` can participate in many contexts without being owned by one chronological notebook. The note becomes a **reusable semantic object**.

Concept maps and meaningful learning

Joseph Novak and Alberto Cañas's concept-mapping theory is especially relevant. A concept map represents concepts plus explicit relationships between them; their framework draws on meaningful-learning theory, in which new concepts become useful when integrated with relevant prior knowledge rather than memorized in isolation. ¹⁷

There is, however, a critical distinction between a **concept map** and Obsidian's default graph.

A formal concept map typically carries meaningful relationship labels:

```
Connection pool
  |
  | exhaustion causes
  v
Database queuing
  |
  | may manifest as
  v
Request timeout
```

Obsidian's graph view primarily visualizes the existence of note links. Obsidian documents global and local graph views as visualizations of notes and their connections, while concept-mapping theory emphasizes that linking phrases identify what the relationship means. ¹⁸

So:

Obsidian's graph is primarily a link graph, not a fully typed semantic graph.

That matters. Ten backlinks do not tell you whether ten sources *support*, *contradict*, *cause*, *depend upon*, *implement*, or merely *mention* the current concept. For rigorous research, the semantic content should remain in prose, properties, link context, or explicit relation conventions rather than being delegated to graph topology.

Sequential notes preserve information networks can destroy

Sequential organization is not an outdated artifact of paper. Order itself can encode causality, process, provenance, and narrative structure.

Research on naturalistic memory provides a particularly good bridge between the two models. Lee and Chen represented narratives as networks and found that semantic and causal centrality predicted memory, but the underlying material was still a **sequence of events**. Network connectedness and event order are complementary rather than competing descriptions. ¹⁹

Similarly, hippocampal research emphasizes temporal/contextual associations in episodic memory. ¹² This supports retaining original meeting notes, reading notes, laboratory observations, and project journals even after extracting their durable ideas.

A useful transformation is therefore:

```
SEQUENTIAL CAPTURE
meeting / paper / lecture / experiment / daily log
      |
      | distill
      v
TOPICAL KNOWLEDGE
concept — concept — claim — method — person
  \                               /
  \----- provenance -----/
```

The backlink to the original source is not clutter. It preserves the context that topical decomposition otherwise destroys.

What the empirical evidence actually says

Concept mapping generally helps, but the mechanism matters

The evidence for graphical/networked organization is positive, but not overwhelmingly one-sided.

Nesbit and Adesope's 2006 meta-analysis examined 55 studies with 5,818 participants involving construction, modification, or viewing of concept/knowledge maps. It found learning benefits across multiple educational domains. ²⁰ A later meta-analysis by Schroeder, Nesbit, Anguiano, and Adesope synthesized 142 independent effects involving 11,814 learners and reported a moderate positive overall effect, approximately $g = 0.58$. ²¹

These findings support **active organization of conceptual relationships**. They do not demonstrate that simply looking at an automatically generated Obsidian graph improves learning.

Indeed, Karpicke and Blunt's Science study found retrieval practice produced stronger meaningful learning than elaborative studying with concept mapping in their experiments. ²² Lechuga and colleagues subsequently reported further evidence favoring repeated retrieval over concept mapping under their tested conditions. ²³

The practical implication is important:

Links are useful when making or traversing them forces meaningful retrieval, comparison, or synthesis. Link accumulation by itself is not a learning strategy.

That argues for notes containing explicit statements such as:

```
## Relationship to [[Connection pooling]]
```

```
The retry storm amplifies pool exhaustion because every timed-out  
request increases concurrent connection demand.
```

rather than mechanically adding five unrelated wikilinks at the bottom.

Hypertext has a real cognitive cost

Networked systems add freedom of navigation, but freedom has a price. DeStefano and LeFevre's review of hypertext reading examined how link structures and navigation demands interact with cognitive load and comprehension. The literature they reviewed showed that navigating nonlinear material can impose additional processing demands and can produce disorientation, particularly when readers must decide repeatedly where to go next. ²⁴

This explains one of the most common failure modes of personal knowledge graphs: a user opens a note, sees twenty possible links, opens four tabs, follows another branch, and eventually loses the original task.

Graph richness and usable knowledge are therefore not monotonically related.

Conceptually:

$$\text{Value of linking} / \text{number of links} =$$

A better approximation is:

$$\text{Value} \approx \frac{\text{meaningful retrieval} + \text{synthesis} + \text{navigation benefit}}{\text{maintenance cost} + \text{ambiguity} + \text{cognitive switching}}$$

That equation is a design heuristic rather than a published cognitive law, but it captures what the empirical literature suggests. ²⁵

Note-taking benefits come from both processing and external storage

A classic distinction in note-taking research separates **encoding** from **external storage**. Encoding refers to cognitive processing performed while taking the note; external storage refers to having an artifact that can later be reviewed and reorganized. Research has found support for both roles, with review often making the stored note especially valuable. ²

This distinction matters for Obsidian because a sophisticated vault can create the illusion that information has been learned merely because it has been stored elegantly. External storage is not internal mastery.

The widely circulated claim that handwriting is necessarily cognitively superior to typing should also be treated carefully. A 2021 direct replication of Mueller and Oppenheimer found no evidence that longhand note taking improved immediate learning through superior encoding, and an exploratory meta-analysis of related experiments did not support a robust longhand advantage. ²⁶

The more stable lesson is therefore **generative processing over transcription**, rather than “paper beats computers” or “graphs beat linear documents.”

Comparative use cases

Dimension	Topical / networked	Sequential / linear
Primary organizing question	“What is this about, and what is it related to?”	“What came before/after this?”
Cognitive analogue	Semantic associations; categories; conceptual relationships. ²⁷	Episodic, temporal, causal and narrative context. ²⁸
Strongest use cases	Literature synthesis, technical reference, research claims, reusable explanations, people/entities, design patterns, evergreen knowledge	Meetings, interviews, experiments, legal/audit histories, daily journals, lectures, incident timelines, procedures
Main retrieval advantage	Multiple paths to the same idea; cross-domain discovery	Natural reconstruction of context and chronology
Main learning advantage	Encourages comparison, integration and explicit relationships when done actively. ²⁹	Preserves original explanatory/ narrative structure and event context. ²⁸
Main failure mode	Link maintenance, fragmentation, graph clutter, navigation overload. ²⁴	Ideas become buried in dates/pages; duplication; weak cross-project reuse
Bad extreme	One tiny note per sentence plus thousands of context-free links	One enormous chronological notebook with no indexes or extracted concepts
Best role in a hybrid system	Long-lived semantic layer	Capture/provenance layer

This leads to a practical operating model for knowledge work:

Capture sequentially → retrieve actively → distill topically → connect deliberately → keep provenance.

That workflow is more defensible from the evidence than choosing either topology exclusively. ³⁰

Obsidian architecture and data model

Obsidian is local-file-oriented rather than database-first

Obsidian's most consequential architectural choice is also one of its simplest: a **vault is an ordinary folder on the local filesystem**, and notes are Markdown-formatted text files. Obsidian identifies `.md` as its primary note format. ³¹

Structured properties are represented in YAML/frontmatter at the beginning of Markdown documents. ³² This creates a data model that is much less opaque than systems in which the canonical note exists only as a row in an application database.

An approximate logical model is:

```
Vault
├─ topic-a.md
├─ topic-b.md
├─ research/
│   └─ paper-a.md
│   └─ paper-b.md
├─ attachments/
│   └─ figure.png
└─ application/plugin configuration
```

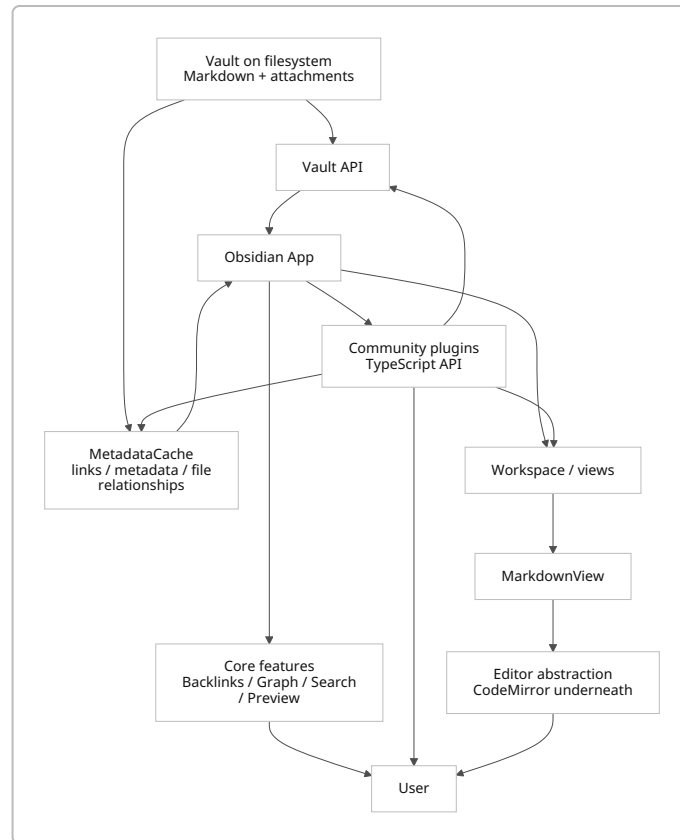
The filesystem is canonical enough that another editor, script, Git, backup utility, indexer, or agent can manipulate the note corpus without reproducing Obsidian's entire application stack. That interoperability follows directly from the vault/folder and Markdown model. ³¹

Public application architecture

Because Obsidian's application implementation is proprietary, its full internal module graph cannot be responsibly reconstructed from source code. The public TypeScript API does, however, expose a useful architectural boundary.

The central `App` object exposes components including the **vault**, **workspace**, and **metadata cache**. ³³ `MarkdownView` exposes the current Markdown file and editor, while Obsidian documents CodeMirror as the underlying editor and provides an `Editor` abstraction through the plugin API. ³⁴

A conservative public-interface architecture therefore looks like this:



This diagram describes the **documented extension-facing architecture**, not a claim about Obsidian's undisclosed internal source tree. ³⁵

Linking, backlinks, transclusion, and the graph

Obsidian supports both its familiar wikilink form:

```
[[Connection pooling]]
```

and ordinary Markdown-style internal links. ³⁶

Backlinks invert the relationship: if note A links to note B, B's backlinks view can expose A as a linked mention. ³⁷ This is more important architecturally than it first appears. It means a note does not need to know all of its incoming contexts; the index can compute them from the corpus.

Embedding/transclusion adds a further layer:

```
![[Connection pooling]]
```

or a subsection/block-oriented variant. Obsidian's embedding syntax reuses internal links with an exclamation mark, allowing material maintained in one file to appear in another context. ³⁸

This makes networked knowledge significantly more practical because it separates **ownership of information** from **places where information is presented**:

```
Canonical note:
[[Retry storms]]

Presented inside:
[[Distributed systems failure modes]]
[[Incident 2026-08-17]]
[[Capacity planning checklist]]
```

One durable concept can therefore participate in multiple sequential or topical documents without copy-and-paste divergence.

Graph view is the visualization layer over these relationships. Obsidian distinguishes a global graph of the vault from a local graph centered around the active note. ³⁹

Why this architecture aligns well with long-term knowledge management

The combination of plain files, links, reverse links, transclusion, properties, and an extension API provides four useful separations:

Separation	Why it matters
Content vs application	The durable note is Markdown rather than an opaque UI object. ³¹
Storage hierarchy vs semantic relationships	A file can live in one folder while linking to arbitrarily many topics. ⁴⁰
Canonical content vs presentation context	Transclusion allows reuse without necessarily duplicating text. ³⁸
Core application vs behavior extensions	Plugins interact through documented application, vault, workspace, view, metadata, and editor APIs. ⁴¹

This is the technical reason Obsidian works unusually well for hybrid note-taking: **chronology can remain in files while semantics are layered on through links and metadata**.

Open-source status

Obsidian itself is **not open source**. Its license explicitly reserves rights in application text, images, and code, while separately emphasizing that users own their own content. ⁴² An Obsidian team response to open-

sourcing requests has historically emphasized that the small team did not consider open-sourcing the application appropriate and argued that durable local files were the important guarantee for user data. ⁴³

The most accurate description is therefore:

Proprietary application core + user-owned local files + documented extension API + unusually open/interoperable content model.

That combination is different from open-source software, even though it avoids some of the data-lock-in problems associated with proprietary note applications. ⁴⁴

Open-source alternatives and code structure

The following repositories reproduce different subsets of the Obsidian model. Star counts and activity observations are a snapshot around **August 18, 2026** and will naturally change.

Repository and feature comparison

Project	URL	Main language(s)	License	Maturity snapshot	Obsidian-like features
Logseq	https://github.com/logseq/logseq	Clojure / ClojureScript	AGPL-3.0	≈44.5k stars; commits observed Aug. 18, 2026. ⁴⁵	Local knowledge graph, Markdown/ Org-oriented files, pages/blocks, wikilinks, backlinks, graph visualization, references, plugin API. Its major difference is that it is block/outliner-first , not Obsidian's primarily page/file-first interaction model. ⁴⁶

Project	URL	Main language(s)	License	Maturity snapshot	Obsidian-like features
SiYuan	https://github.com/siyuan-note/siyuan	Go + TypeScript/web stack	AGPL-3.0	≈45.9k stars; v3.8.1/ repository commits observed Aug. 18, 2026. ⁴⁷	Block references, two-way links, WYSIWYG Markdown editing, APIs/plugins, graph-oriented PKM functionality and built-in AI. Key difference: canonical document data is <code>.sy</code> JSON rather than ordinary <code>.md</code> , although Markdown export is supported. ⁴⁸
TriliumNext Notes	https://github.com/TriliumNext/Trilium	TypeScript / JavaScript	AGPL-3.0	≈37k stars in the current repository snapshot; active 2026 development. The GitHub newest-commit timestamp did not render reliably in this crawl. ⁴⁹	Deep note hierarchy, note cloning, rich-text and code editors, relation/link maps, scripting, REST API, Canvas and mind maps. It is tree/ database-oriented rather than a plain-Markdown vault clone. ⁵⁰

Project	URL	Main language(s)	License	Maturity snapshot	Obsidian-like features
SilverBullet	https://github.com/silverbulletmd/silverbullet	TypeScript + Rust	MIT	≈5.9k stars; commits observed Aug. 18, 2026. ⁵¹	Markdown files, wikilinks, bidirectional links, CodeMirror 6 editing, queryable objects, Lua programmability, plugs/plugin APIs, and graph exploration. It is one of the strongest fully OSS candidates for building an Obsidian-like programmable substrate. ⁵²
Foam	https://github.com/foambubble/foam	TypeScript	MIT	≈17.4k stars; latest observed main-branch commit Aug. 13, 2026. ⁵³	Graph visualization, wikilinks, rename-aware links, backlinks, embeds/transclusion, sections, aliases, templates, tags, daily notes and Markdown workspace storage. It explicitly notes that an Obsidian vault can usually serve as a Foam workspace. ⁵⁴

Project	URL	Main language(s)	License	Maturity snapshot	Obsidian-like features
Joplin	https://github.com/laurent22/joplin	TypeScript / JavaScript, React/React Native ecosystem	AGPL-3.0-or-later by default	≈56.0k stars; commits observed Aug. 18, 2026. ⁵⁵	Markdown notes, offline-first operation, plugins/themes, full-text search and mature desktop/mobile clients. Less Obsidian-like in storage architecture: it organizes notes through its application data model rather than making a Markdown directory the primary vault abstraction; graph functionality is less central. ⁵⁶
Dendron	https://github.com/dendronhq/dendron	TypeScript	Apache-2.0	≈7.5k stars; project states that active development has ceased and it is maintenance-only. ⁵⁷	Local-first Markdown, wikilink-oriented PKM, hierarchical naming, VS Code integration and extensible tooling. Architecturally interesting but not a sensible foundation for a new product requiring an actively evolving upstream. ⁵⁸

Which are architecturally closest?

Foam is the closest conceptual match if the criterion is “*ordinary Markdown files plus wikilinks/backlinks/graph/transclusion.*” Its README explicitly supports graph visualization, note embedding, backlinks, aliases and Markdown documents, and says existing Obsidian vaults commonly work as Foam workspaces. ⁵⁴ The major trade-off is that Foam delegates the application/editor/plugin shell to VS Code rather than providing a standalone Obsidian-like application.

SilverBullet is arguably the most interesting foundation for developers building something new. It combines a browser-facing Markdown editor with CodeMirror 6, a server component, wiki-style links, bidirectional links, queryable objects, Lua programmability and a separate plug API. ⁵⁹ Its architecture is substantially more inspectable than Obsidian's because the entire platform is OSS.

Logseq has a very mature graph-oriented ecosystem but makes blocks first-class. Its README identifies Clojure/ClojureScript, DataScript, mldoc, isomorphic-git and SCI among the technologies used. ⁶⁰ That makes it a useful reference implementation for block identity, graph querying, outliner semantics, and plugin execution, but not a drop-in architectural analogue of Obsidian.

SiYuan is particularly valuable if the desired product is a structured block database with Markdown-like authoring rather than Markdown being the canonical persistence representation. The repository documents `.sy` JSON document data in the workspace, alongside assets, snippets, templates, widgets and plugins. ⁴⁸

Code-structure comparison

Project	High-level repository structure	Architectural lesson
Logseq	Repository includes <code>src/</code> , <code>deps/</code> , <code>libs/</code> , <code>cli/</code> , <code>android/</code> , <code>sidecar/</code> , <code>resources/</code> , <code>scripts/</code> and related modules. The project uses Clojure/ClojureScript with DataScript, mldoc and SCI among its core technologies. ⁶¹	Good reference for graph/index layers, structured blocks, multi-client code and sandboxable extensibility.
SiYuan	Top-level areas include <code>app/</code> , <code>kernel/</code> , <code>docs/</code> , <code>scripts/</code> and related resources; its ecosystem identifies separate components such as the Lute editor engine and Petal plugin API. ⁴⁸	Strong example of separating a native/Go “kernel” from UI/editor and plugin surfaces.
TriliumNext	Monorepo centers on <code>apps/</code> , <code>packages/</code> , <code>docs/</code> , <code>patches/</code> , and <code>scripts/</code> . The documented stack uses CKEditor 5 for rich text, CodeMirror for code, plus specialized visualization components such as jsPlumb for relation/link maps. ⁶²	Useful example of a component-rich note platform whose semantic model is richer than filesystem Markdown.
SilverBullet	Explicitly separated into <code>client/</code> , <code>server/</code> , <code>server-common/</code> , <code>server-runtime-chrome/</code> , <code>plugins/</code> , <code>libraries/</code> , <code>plug-api/</code> , plus command binaries. Front end is TypeScript/CodeMirror/Preact; server components are Rust. ⁶³	Probably the clearest OSS reference for separating editor UI, storage/server, extension API and user-programmable logic.

Project	High-level repository structure	Architectural lesson
Foam	Monorepo centers on <code>packages/</code> , with <code>docs/</code> , <code>scripts/</code> , assets and VS Code development infrastructure around it. ⁵⁴	Good model for implementing a knowledge graph as an extension over an existing editor/workspace platform.
Joplin	Top-level repository is largely a <code>packages/</code> monorepo with shared app/server/CLI components and platform resources. Its license specifically references package-specific licensing under directories such as <code>packages/server</code> and <code>packages/app-cli</code> . ⁶⁴	Valuable reference for multi-platform mature synchronization, plugin and application infrastructure rather than for vault semantics.
Dendron	<code>packages/</code> , <code>docs/</code> , <code>bootstrap/</code> , <code>dev/</code> , <code>playbooks/</code> , <code>test-workspace/</code> , <code>vendor/</code> and multiple VS Code workspace definitions. ⁶⁵	Historically useful reference for making PKM behave more like a developer IDE; upstream is now maintenance-only. ⁵⁸

For someone implementing an open Obsidian analogue, I would study **SilverBullet for platform boundaries**, **Foam for Markdown-link semantics**, **Logseq for block/graph indexing**, **SiYuan for kernel/UI separation**, and **Joplin for production-grade multiplatform concerns** rather than cloning any one repository wholesale.

AI ecosystem around Obsidian-like tools

AI integration has evolved into several distinct categories: deterministic formatting, generative writing, semantic retrieval, conversational retrieval-augmented generation, and tool-using/autonomous agents.

Current integrations

Project / plugin	URL	Category	What it provides
Copilot for Obsidian	https://github.com/logancyang/obsidian-copilot	Agentic assistant	Current V4 describes itself as “agents for your Obsidian vault” and supports opencode, Claude Code, and Codex integrations. Agents can inspect notes, use tools, create files and continue multi-step tasks subject to permissions. The project also supports reusable projects, skills and commands. ⁶⁶

Project / plugin	URL	Category	What it provides
Smart Connections	https://github.com/brianpetro/obsidian-smart-connections	Semantic retrieval / AI connections	Uses local embeddings and a “Smart Environment” to surface semantically related notes rather than relying only on explicit wikilinks. This is effectively a learned semantic edge layer alongside Obsidian's manually authored graph. ⁶⁷
Text Generator	https://github.com/nhaouari/obsidian-textgenerator-plugin	Generation / transformation	Supports text generation using multiple AI providers, including cloud providers and local models, making it useful for summarization, rewriting and template-driven generation. ⁶⁸
Obsidian Linter	https://github.com/platers/obsidian-linter	Deterministic formatting	Formats and styles notes through configurable rules. It is not an LLM plugin, which is precisely why it is important: many “AI formatting” tasks should be handled deterministically. ⁶⁹
Khoj	https://github.com/khoj-ai/khoj	Personal AI / autonomous research	Open-source and self-hostable; works with Obsidian among other clients, can use local or online LLMs, search personal documents and the internet, create custom tool-using agents, and automate recurring research. ⁷⁰
mcp-logseq	GitHub project surfaced for Logseq MCP integration	Agent bridge	Connects external AI assistants to a Logseq graph through the application's local API, illustrating the model in which the note app exposes tools and an external agent owns the reasoning loop. ⁷¹
SiYuan built-in AI	https://github.com/siyuan-note/siyuan	Native writing/ chat/agent functionality	SiYuan documents AI writing and Q&A through model APIs, and its 2026 repository activity includes expanding agent-oriented AI functionality. ⁴⁷

Three useful AI architectures

The first pattern is **inline transformation**:

```

selected Markdown
    ↓
prompt + style rules
    ↓

```

```

LLM
  ↓
proposed replacement
  ↓
diff / accept

```

This is appropriate for “turn these rough bullets into clean Markdown,” “normalize headings,” “extract decisions,” or “convert prose into a checklist.”

The second is **semantic retrieval**:

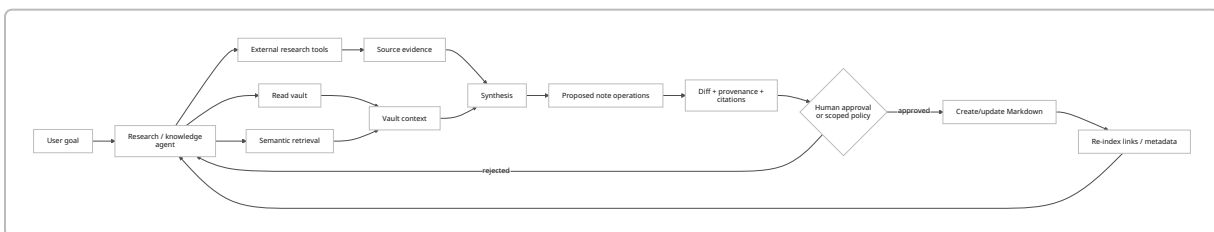
```

vault files
  ↓
chunk / embed / index
  ↓
vector similarity
  ↓
related notes
  ↓
optional LLM synthesis

```

Smart Connections exemplifies this direction by using embeddings to surface semantically related material rather than requiring every connection to have been manually linked. ⁷²

The third is **agentic knowledge work**:



Current Copilot for Obsidian explicitly supports multi-step agents that inspect notes, use tools, and create files, while Khoj supports custom agents, internet/document retrieval, scheduled automation and self-hosting. ⁷³

AI changes the meaning of the knowledge graph

There are now potentially three kinds of edges in an AI-enabled vault:

```

Explicit human edge:
A --[[wikilink]]--> B

```

Structural edge:

A --same property/tag/project--> B

Inferred AI edge:

A --embedding similarity 0.86--> B

These should not be conflated.

A manually written link has relatively high epistemic value because somebody intentionally asserted a relationship. A tag/property relationship is structured but may be broad. An embedding similarity is probabilistic: it says two texts are close in a learned representational space, not that one supports or causes the other.

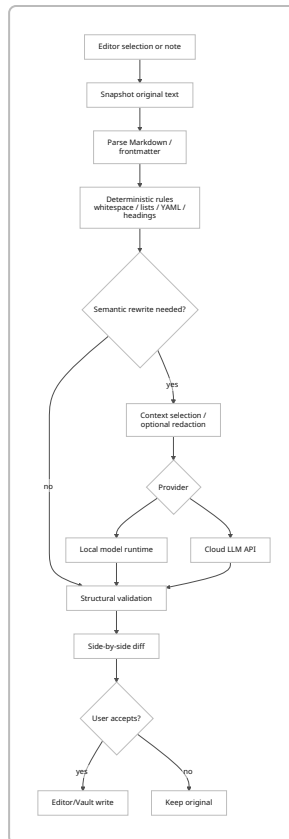
The most useful future knowledge tools will likely expose all three while visually distinguishing them.

That is particularly important for research. An autonomous agent should never silently turn “semantically similar” into “evidence supporting.”

Implementing an AI Markdown formatter and research agent

Recommended formatter architecture

A robust formatter should be **parser-first, LLM-second**.



The Obsidian side can be implemented as a TypeScript community plugin. The documented editor API gives plugins an abstraction over the underlying CodeMirror editor, while `MarkdownView` provides access to the active note/file context. ³⁴

A reasonable implementation stack is:

```

Obsidian Plugin API
+
Markdown parser/AST
+
deterministic formatter/linter
+
LLM-provider abstraction
+
structure validator
+
diff/approval UI
  
```

For the deterministic layer, suitable ecosystem choices include `unified` / `remark`, Prettier's Markdown handling, markdownlint, or code adapted from the OSS Obsidian Linter project. The important design choice is not the exact library: it is to avoid paying an LLM to make changes that a deterministic rule can make more safely.

Preserve Obsidian-specific syntax explicitly

A generic Markdown model may “clean up” syntax that is semantically important to Obsidian. The formatter’s contract should explicitly preserve:

```
---
project: knowledge-tooling
status: active
---

[[Internal links]]
[[Target|Aliases]]
![[Transcluded note]]
![[figure.png]]

#tag
#nested/tag

^block-id

> [!NOTE]
> Obsidian callout content

```language

literal code that must not be rewritten
```

Obsidian officially supports both wikilink and standard Markdown internal-link forms, stores properties in YAML, and implements links/backlinks as first-class application features. <sup>74</sup>

A safe AI prompt should therefore look more like this than “make my Markdown prettier”:

```
SYSTEM:
You format Obsidian Markdown.

Your task is organizational, not factual.

Hard constraints:

- Preserve every factual claim unless the user explicitly requests rewriting.
- Preserve YAML keys and values unless a requested transformation targets them.
- Preserve [[wikilinks]], aliases, embeds, block IDs, tags, URLs, citations,
 code fences, math, HTML comments, and callout markers exactly.

- Never invent links, sources, citations, dates, tags, or metadata.
```

- You may normalize headings, lists, whitespace, paragraph breaks, and presentation structure.
- Do not wrap the result in a Markdown code fence.
- Return only the transformed document.

USER:

Formatting profile:

- ATX headings
- one blank line around headings
- "-" list marker
- retain original wording where possible

DOCUMENT:

<document>

...

</document>

For a research formatter, add:

Any sentence whose factual meaning would change must remain unchanged.  
If a requested structure cannot be produced without changing factual meaning, leave that passage untouched.

## Minimal TypeScript plugin skeleton

The following is intentionally provider-neutral. The LLM implementation can point to a local runtime, an enterprise gateway, or a cloud service without coupling the editor command to one vendor.

```
import { Editor, MarkdownView, Notice, Plugin } from "obsidian";

interface FormatterResult {
 markdown: string;
 warnings: string[];
}

interface MarkdownFormatter {
 format(markdown: string): Promise<FormatterResult>;
}

export default class AiMarkdownFormatterPlugin extends Plugin {
 // Inject a real implementation during onload/configuration.
 private formatter!: MarkdownFormatter;
}
```

```

async onload(): Promise<void> {
 this.formatter = createFormatterFromSettings();

 this.addCommand({
 id: "ai-format-markdown",
 name: "Format Markdown with AI",
 editorCallback: async (editor: Editor, view: MarkdownView) => {
 const selected = editor.getSelection();
 const wholeDocument = selected.length === 0;
 const original = wholeDocument ? editor.getValue() : selected;

 if (!original.trim()) {
 new Notice("Nothing to format.");
 return;
 }

 try {
 const result = await this.formatter.format(original);

 const validation = validateObsidianStructure(original,
result.markdown);

 if (!validation.safe) {
 new Notice(
 `Formatting stopped: ${validation.reasons.join("; ")}`
);
 return;
 }

 // Production version:
 // show a diff modal here and require explicit acceptance.
 const accepted = await showDiffAndConfirm(
 original,
 result.markdown,
 result.warnings
);

 if (!accepted) return;

 if (wholeDocument) {
 editor.setValue(result.markdown);
 } else {
 editor.replaceSelection(result.markdown);
 }

 new Notice("Markdown formatted.");
 } catch (error) {
 console.error("AI formatter failed", error);
 }
 }
 });
}

```

```

 new Notice("Formatting failed; the original text was not changed.");
 }
},
});
}
}

```

The relevant Obsidian concepts here—plugin commands operating against `Editor` / `MarkdownView`, and the public editor abstraction—are exposed in Obsidian's documented plugin API. <sup>34</sup>

The validator should compare protected constructs before allowing the write:

```

type ValidationResult =
 | { safe: true; reasons: [] }
 | { safe: false; reasons: string[] };

function validateObsidianStructure(
 before: string,
 after: string
): ValidationResult {
 const reasons: string[] = [];

 const protectedPatterns = {
 wikilinks: /\[\[^\]]+\]\]/g,
 embeds: /!\[\[^\]]+\]\]/g,
 blockIds: /\^[A-Za-z0-9-]+/g,
 urls: /https?:\/\/[^\s)]+/g,
 codeFences: /```.*$/gm,
 };

 for (const [name, pattern] of Object.entries(protectedPatterns)) {
 const a = [...before.matchAll(pattern)].map((m) => m[0]).sort();
 const b = [...after.matchAll(pattern)].map((m) => m[0]).sort();

 if (JSON.stringify(a) !== JSON.stringify(b)) {
 reasons.push(`${name} changed`);
 }
 }

 // A production version should parse YAML and Markdown ASTs rather than
 // relying only on regular expressions.
 return reasons.length
 ? { safe: false, reasons }
 : { safe: true, reasons: [] };
}

```



The regex layer is only an inexpensive guard. A production implementation should parse frontmatter and Markdown structurally before comparing protected nodes.

## Provider abstraction

Do not embed one LLM vendor throughout the application. Treat inference as an adapter:

```
interface LlmProvider {
 complete(request: {
 system: string;
 user: string;
 temperature?: number;
 }): Promise<string>;
}

class AiFormatter implements MarkdownFormatter {
 constructor(private readonly llm: LlmProvider) {}

 async format(markdown: string): Promise<FormatterResult> {
 const system = buildFormattingContract();
 const user = buildFormattingRequest(markdown);

 const output = await this.llm.complete({
 system,
 user,
 temperature: 0,
 });

 return {
 markdown: stripAccidentalCodeFence(output),
 warnings: [],
 };
 }
}
```

This permits:

```
LlmProvider
├─ LocalProvider
│ ├── local server
│ └─ on-device model runtime
├─ CloudProvider
│ ├── organization gateway
│ └─ public model API
```

```
└─ MockProvider
 └─ unit tests
```

Text Generator's support for multiple AI providers and local models shows that a provider-agnostic architecture is already practical in the Obsidian ecosystem. <sup>68</sup>

## Offline versus cloud

Dimension	Local/offline model	Cloud model
<b>Note-content privacy</b>	Content can remain on the user's own machine if the entire inference path is local. Khoj explicitly supports private self-hosting/local operation, and current Obsidian AI plugins also expose local-model paths. <sup>75</sup>	Selected note/context is transmitted outside the machine; treatment then depends on the provider and account agreement.
<b>Latency</b>	Avoids network round trips but depends on local hardware/model size	Often strong inference throughput; network latency applies
<b>Model capability</b>	Can be excellent for formatting, extraction and many RAG tasks; resource limits constrain model size	Easier access to large frontier models
<b>Offline availability</b>	Yes	Usually no
<b>Deployment burden</b>	Model installation, updates, memory/GPU/CPU management	API credentials, cost controls and network configuration
<b>Best privacy strategy</b>	Local model + local embeddings + no telemetry	Minimize transmitted context, explicitly disclose transfer, redact secrets, and support a "never send entire vault" policy

An important implementation detail follows from Obsidian's plugin storage model: plugin settings are data written to disk through the plugin API, not automatically a hardware-backed secret vault. <sup>76</sup> Security-sensitive implementations should therefore avoid treating ordinary plugin settings as equivalent to an operating-system credential store.

## Autonomous research-agent design

For research, the formatter becomes one tool in a larger system.

A practical tool interface might expose:

```
vault.list_files()
vault.read_note(path)
```

```
vault.search(query)
vault.get_backlinks(path)
vault.create_note(path, content)
vault.patch_note(path, patch)
research.search_web(query)
research.fetch_source(id)
research.extract_citation(id)
markdown.validate(content)
markdown.format(content)
```

The agent should receive **read permissions much more freely than write permissions**.

A safe policy is:

```
Read:
 /Research/**
 /Concepts/**
 /Sources/**
 allowed automatically

Write:
 /Agent Drafts/**
 allowed automatically

Modify existing canonical notes:
 requires diff approval

Delete / rename:
 always requires approval

External transmission:
 only explicitly selected text or retrieved context

Source claims:
 citation required before promotion from draft to canonical note
```

That architecture has an additional knowledge-management benefit: it separates **machine-proposed structure from human-endorsed knowledge**.

A research flow could create:

```
/Sources/
 paper-smith-2025.md
 web-source-foo.md
```

```
/Agent Drafts/
 synthesis-semantic-memory.md
```

```
/Concepts/
 semantic-memory.md
```

The first layer preserves evidence, the second is provisional synthesis, and only the third represents accepted long-lived knowledge.

Current Copilot for Obsidian's permission-controlled multi-step agents and Khoj's custom agents/research automation demonstrate that this architecture has already moved from hypothetical design into deployed tooling. <sup>73</sup>

## Conclusions and source index

The core design lesson is not that “brains are graphs, therefore notes should be graphs.” That would overstate both neuroscience and knowledge-management research.

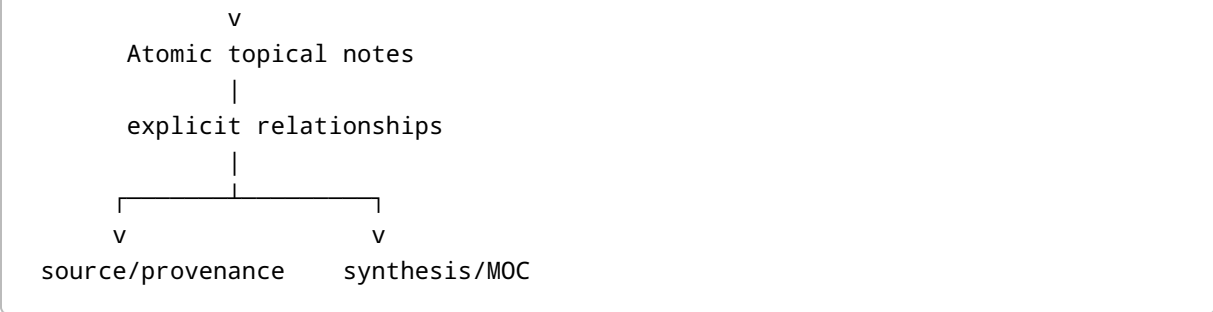
A more defensible chain is:

```
Human semantic knowledge is strongly associative.
 ↓
Explicitly relating concepts can support meaningful organization.
 ↓
Networked notes make those associations externally navigable.
 ↓
But human experience also has temporal, causal and narrative structure.
 ↓
Therefore retain sequential records and distill reusable concepts from them.
 ↓
Obsidian is well suited to this hybrid because file hierarchy,
chronology, wikilinks, backlinks, metadata and transclusion coexist.
```

The semantic-network premise is supported by cognitive research that represents concepts and associations as networks; concept-mapping meta-analyses provide evidence that explicit conceptual organization can improve learning. <sup>3</sup> The counterweight is equally important: episodic memory encodes spatiotemporal context, hypertext can impose cognitive-load costs, and retrieval practice can outperform elaborate mapping. <sup>77</sup>

For **knowledge architecture**, the highest-value pattern is therefore:

```
Inbox / Daily / Meeting / Reading notes
 |
 | review + active retrieval
```



For **software architecture**, Obsidian is compelling because it puts durable Markdown files underneath a rich indexing and extension layer, but its proprietary core limits source-level customization. <sup>78</sup> An OSS implementation should look especially closely at SilverBullet and Foam for an Obsidian-like file/link/editor model, with Logseq, SiYuan, TriliumNext and Joplin contributing valuable patterns for blocks, kernels, relational note models, synchronization and multiplatform application design. <sup>79</sup>

For **AI**, the most robust direction is not to let an LLM become the canonical storage layer. The canonical layer should remain inspectable Markdown or another explicit data model; AI should operate as a retrieval, transformation and proposal layer around it. Deterministic formatting should precede LLM formatting, protected syntax should be structurally validated, and autonomous writes should be diffed and permission-scoped. Existing tools such as Obsidian Linter, Text Generator, Smart Connections, Copilot for Obsidian, and Khoj collectively demonstrate each layer of that stack. <sup>80</sup>

Primary and high-value source links

Topic	Source
Obsidian storage model	<a href="#">Obsidian Help — How Obsidian stores data</a> <sup>81</sup>
Obsidian internal links	<a href="#">Obsidian Help — Internal links</a> <sup>82</sup>
Obsidian backlinks	<a href="#">Obsidian Help — Backlinks</a> <sup>83</sup>
Obsidian Graph view	<a href="#">Obsidian Help — Graph view</a> <sup>84</sup>
Obsidian application API	<a href="#">Obsidian Developer Docs — App</a> <sup>33</sup>
Obsidian editor API	<a href="#">Obsidian Developer Docs — Editor</a> <sup>85</sup>
Obsidian licensing	<a href="#">Obsidian License Overview</a> <sup>42</sup>
Concept-map theory	<a href="#">IHMC — The Theory Underlying Concept Maps</a> <sup>86</sup>
Concept-map meta-analysis	<a href="#">Nesbit &amp; Adesope, 2006, Review of Educational Research</a> <sup>20</sup>
Later concept-map meta-analysis	<a href="#">Schroeder et al., Educational Psychology Review, DOI 10.1007/s10648-017-9403-9</a> <sup>87</sup>
Semantic-memory networks	<a href="#">Ovando-Tellez et al., Communications Biology, 2022</a> <sup>14</sup>
Networks and naturalistic memory	<a href="#">Lee &amp; Chen, Nature Communications, 2022</a> <sup>19</sup>

Topic	Source
Episodic/spatiotemporal memory	<a href="#">Liu et al., Science, 2023</a> <sup>12</sup>
Retrieval versus concept mapping	<a href="#">Karpicke &amp; Blunt, Science, 2011 — PubMed record</a> <sup>22</sup>
Hypertext cognitive load	<a href="#">DeStefano &amp; LeFevre, Computers in Human Behavior</a> <sup>24</sup>
Laptop/longhand replication	<a href="#">Urry et al., Psychological Science, 2021</a> <sup>88</sup>
Logseq	<a href="https://github.com/logseq/logseq">https://github.com/logseq/logseq</a> <sup>60</sup>
SiYuan	<a href="https://github.com/siyuan-note/siyuan">https://github.com/siyuan-note/siyuan</a> <sup>89</sup>
TriliumNext Notes	<a href="https://github.com/TriliumNext/Trilium">https://github.com/TriliumNext/Trilium</a> <sup>90</sup>
SilverBullet	<a href="https://github.com/silverbulletmd/silverbullet">https://github.com/silverbulletmd/silverbullet</a> <sup>91</sup>
Foam	<a href="https://github.com/foambubble/foam">https://github.com/foambubble/foam</a> <sup>54</sup>
Joplin	<a href="https://github.com/laurent22/joplin">https://github.com/laurent22/joplin</a> <sup>56</sup>
Dendron	<a href="https://github.com/dendronhq/dendron">https://github.com/dendronhq/dendron</a> <sup>58</sup>
Copilot for Obsidian	<a href="https://github.com/logancyang/obsidian-copilot">https://github.com/logancyang/obsidian-copilot</a> <sup>66</sup>
Smart Connections	<a href="https://github.com/brianpetro/obsidian-smart-connections">https://github.com/brianpetro/obsidian-smart-connections</a> <sup>72</sup>
Text Generator	<a href="https://github.com/nhaouari/obsidian-textgenerator-plugin">https://github.com/nhaouari/obsidian-textgenerator-plugin</a> <sup>68</sup>
Obsidian Linter	<a href="https://github.com/platers/obsidian-linter">https://github.com/platers/obsidian-linter</a> <sup>69</sup>
Khoj	<a href="https://github.com/khoj-ai/khoj">https://github.com/khoj-ai/khoj</a> <sup>70</sup>

<sup>1</sup> <sup>3</sup> <sup>14</sup> <sup>27</sup> <https://www.nature.com/articles/s42003-022-03547-x>  
<https://www.nature.com/articles/s42003-022-03547-x>

<sup>2</sup> <sup>30</sup> <https://www.sciencedirect.com/science/article/pii/S0361476X78900206>  
<https://www.sciencedirect.com/science/article/pii/S0361476X78900206>

<sup>4</sup> <sup>12</sup> <sup>77</sup> <https://www.science.org/doi/10.1126/science.adi8237>  
<https://www.science.org/doi/10.1126/science.adi8237>

<sup>5</sup> <sup>24</sup> <sup>25</sup> <https://www.sciencedirect.com/science/article/pii/S0747563205000658>  
<https://www.sciencedirect.com/science/article/pii/S0747563205000658>

<sup>6</sup> <sup>31</sup> <sup>78</sup> <sup>81</sup> **How Obsidian stores data**  
[https://obsidian.md/help/data-storage?utm\\_source=chatgpt.com](https://obsidian.md/help/data-storage?utm_source=chatgpt.com)

<sup>7</sup> <sup>42</sup> <sup>44</sup> **License Overview**  
[https://obsidian.md/license?utm\\_source=chatgpt.com](https://obsidian.md/license?utm_source=chatgpt.com)

8 53 54 <https://github.com/foambubble/foam>  
<https://github.com/foambubble/foam>

9 66 73 <https://github.com/logancyang/obsidian-copilot>  
<https://github.com/logancyang/obsidian-copilot>

10 69 80 <https://github.com/platers/obsidian-linter>  
<https://github.com/platers/obsidian-linter>

11 13 **A Spreading-Activation Theory of Semantic Processing**  
[https://philpapers.org/rec/COLAST-3?utm\\_source=chatgpt.com](https://philpapers.org/rec/COLAST-3?utm_source=chatgpt.com)

15 <https://www.nature.com/articles/s41598-022-11698-4>  
<https://www.nature.com/articles/s41598-022-11698-4>

16 <https://www.nature.com/articles/s44271-024-00100-w>  
<https://www.nature.com/articles/s44271-024-00100-w>

17 86 <https://cmap.ihmc.us/docs/theory-of-concept-maps>  
<https://cmap.ihmc.us/docs/theory-of-concept-maps>

18 39 84 **Graph view - Obsidian Help**  
[https://obsidian.md/help/plugins/graph?utm\\_source=chatgpt.com](https://obsidian.md/help/plugins/graph?utm_source=chatgpt.com)

19 28 <https://www.nature.com/articles/s41467-022-31965-2>  
<https://www.nature.com/articles/s41467-022-31965-2>

20 29 <https://journals.sagepub.com/doi/10.3102/00346543076003413>  
<https://journals.sagepub.com/doi/10.3102/00346543076003413>

21 [https://www.researchgate.net/publication/315507389\\_Studying\\_and\\_Constructing\\_Concept\\_Maps\\_a\\_Meta-Analysis](https://www.researchgate.net/publication/315507389_Studying_and_Constructing_Concept_Maps_a_Meta-Analysis)  
[https://www.researchgate.net/publication/315507389\\_Studying\\_and\\_Constructing\\_Concept\\_Maps\\_a\\_Meta-Analysis](https://www.researchgate.net/publication/315507389_Studying_and_Constructing_Concept_Maps_a_Meta-Analysis)

22 **Retrieval practice produces more learning than elaborative ...**  
[https://pubmed.ncbi.nlm.nih.gov/21252317/?utm\\_source=chatgpt.com](https://pubmed.ncbi.nlm.nih.gov/21252317/?utm_source=chatgpt.com)

23 **Further evidence that concept mapping is not better than ...**  
[https://www.sciencedirect.com/science/article/abs/pii/S0959475215300232?utm\\_source=chatgpt.com](https://www.sciencedirect.com/science/article/abs/pii/S0959475215300232?utm_source=chatgpt.com)

26 <https://pubmed.ncbi.nlm.nih.gov/33539228/>  
<https://pubmed.ncbi.nlm.nih.gov/33539228/>

32 **Properties - Obsidian Help**  
[https://obsidian.md/help/properties?utm\\_source=chatgpt.com](https://obsidian.md/help/properties?utm_source=chatgpt.com)

33 35 41 **App - Developer Documentation**  
[https://docs.obsidian.md/Reference/TypeScript%2BAPI/App?utm\\_source=chatgpt.com](https://docs.obsidian.md/Reference/TypeScript%2BAPI/App?utm_source=chatgpt.com)

34 **MarkdownView - Developer Documentation**  
[https://docs.obsidian.md/Reference/TypeScript%2BAPI/MarkdownView?utm\\_source=chatgpt.com](https://docs.obsidian.md/Reference/TypeScript%2BAPI/MarkdownView?utm_source=chatgpt.com)

36 74 <https://obsidian.md/help/syntax>  
<https://obsidian.md/help/syntax>

37 83 <https://obsidian.md/help/plugins/backlinks>  
<https://obsidian.md/help/plugins/backlinks>

38 Embed files - Obsidian Help

[https://obsidian.md/help/How%2Bto/Embed%2Bfiles?utm\\_source=chatgpt.com](https://obsidian.md/help/How%2Bto/Embed%2Bfiles?utm_source=chatgpt.com)

40 82 <https://obsidian.md/help/links>

<https://obsidian.md/help/links>

43 Open Sourcing of Obsidian - Meta

[https://forum.obsidian.md/t/open-sourcing-of-obsidian/1515?utm\\_source=chatgpt.com](https://forum.obsidian.md/t/open-sourcing-of-obsidian/1515?utm_source=chatgpt.com)

45 46 60 61 <https://github.com/logseq/logseq>

<https://github.com/logseq/logseq>

47 48 89 <https://github.com/siyuan-note/siyuan>

<https://github.com/siyuan-note/siyuan>

49 50 62 90 <https://github.com/TriliumNext/trilium>

<https://github.com/TriliumNext/trilium>

51 52 59 63 79 91 <https://github.com/silverbulletmd/silverbullet>

<https://github.com/silverbulletmd/silverbullet>

55 56 64 <https://github.com/laurent22/joplin>

<https://github.com/laurent22/joplin>

57 58 65 <https://github.com/dendronhq/dendron>

<https://github.com/dendronhq/dendron>

67 72 <https://github.com/brianpetro/obsidian-smart-connections>

<https://github.com/brianpetro/obsidian-smart-connections>

68 <https://github.com/nhaouari/obsidian-textgenerator-plugin>

<https://github.com/nhaouari/obsidian-textgenerator-plugin>

70 75 <https://github.com/khoj-ai/khoj>

<https://github.com/khoj-ai/khoj>

71 <https://github.com/ergut/mcp-logseq>

<https://github.com/ergut/mcp-logseq>

76 Settings - Developer Documentation

[https://docs.obsidian.md/Plugins/User%2Binterface/Settings?utm\\_source=chatgpt.com](https://docs.obsidian.md/Plugins/User%2Binterface/Settings?utm_source=chatgpt.com)

85 Editor - Developer Documentation

[https://docs.obsidian.md/Plugins/Editor/Editor?utm\\_source=chatgpt.com](https://docs.obsidian.md/Plugins/Editor/Editor?utm_source=chatgpt.com)

87 <https://rex.libraries.wsu.edu/esploro/outputs/journalArticle/Studying-and-Constructing-Concept-Maps-a/99900601155801842>

<https://rex.libraries.wsu.edu/esploro/outputs/journalArticle/Studying-and-Constructing-Concept-Maps-a/99900601155801842>

88 <https://journals.sagepub.com/doi/10.1177/0956797620965541>

<https://journals.sagepub.com/doi/10.1177/0956797620965541>